

PLANO ARQUITETURAL: NOVA FEATURE AIDD-OPS

Meta-Orquestrador Agêntico Universal e Agnóstico para Stacks Open Source & White-Label
Governança Determinística pelo Ecosistema AIDD e Síntese Dinâmica de Infraestrutura

Documento: plano_nova_feature_aidd_ops.docx	Natureza: Agnóstico a Ferramentas, Stacks e Nichos de Mercado
Engenharia Agêntica: Bifurcação Estrita: Inferência Probabilística vs. Execução Determinística	Governança: Ecosistema AIDD (Multi-Harness, MCPs & Quality Gates)

1. Sumário Executivo: A Natureza Agnóstica do AIDD-Ops

A feature AIDD-Ops estende o ecossistema-aidd para o domínio de operações de infraestrutura, atuando como um Meta-Orquestrador Agêntico Universal e Agnóstico de Stacks Open Source sob medida. O AIDD-Ops NÃO foi projetado para atender a uma ferramenta ou nicho proprietário específico (como o GoHighLevel). O GoHighLevel é citado ao longo desta especificação estritamente como um estudo de caso ilustrativo e prova de conceito (PoC) de como uma plataforma SaaS monolítica, fechada e cara pode ser decomposta e substituída por uma arquitetura aberta, componível e de alta performance.

O propósito fundamental do AIDD-Ops é a independência absoluta de ferramentas: o sistema é capaz de receber qualquer conjunto de dores de negócio em linguagem natural — seja para operações de saúde, indústria B2B, e-commerce, logística, fintechs, plataformas educacionais ou pipelines de inteligência artificial — e autonomamente descobrir, validar, dimensionar, integrar e implantar a stack open source mais eficiente para atender àquela demanda específica.

2. As 5 Camadas Arquiteturais Fundamentais

Independentemente das ferramentas open source selecionadas para a stack final, a arquitetura obedece a um padrão universal em 5 camadas:

Camada	Tecnologia Principal	Responsabilidade Técnica
Camada 1: Borda e Roteamento	Traefik / Nginx Proxy Manager	Recepção HTTPS na porta 443, provisionamento e renovação automática de certificados SSL via Let's Encrypt, terminação TLS e roteamento por subdomínios (app., api., etc.).
Camada 2: Acesso e Identidade	Authentik / Keycloak	Single Sign-On (SSO) corporativo via

(SSO)		OIDC/OAuth2 e SAML. Gestão centralizada de usuários, MFA e segregação lógica entre organizações.
Camada 3: Apresentação White-Label	Next.js + TypeScript + Tailwind CSS + shadcn/ui	Frontend unificado com Backend for Frontend (BFF). Aplica dinamicamente cores e logotipo do cliente. Consome dados via APIs REST/GraphQL e embute módulos via iframes seguros.
Camada 4: Execução Modular & Banco Central	Docker Engine & PostgreSQL Centralizado	Microserviços independentes da stack escolhida conectados a 1 ÚNICA instância centralizada de PostgreSQL com bancos lógicos dinamicamente provisionados.
Camada 5: Orquestração e Barramento	Micro-Serviço Gateway Nativo (FastAPI)	Sistema nervoso central e padrão arquitetural nativo do AIDD-Ops. Micro-serviço assíncrono em Python/FastAPI (30-50 MB RAM) com tipagem Pydantic estrita, testes automatizados e controle Git-native. (O n8n atua apenas como add-on visual opcional).

3. A Squad Agêntica e a Fronteira entre Probabilístico e Determinístico

Um dos conceitos nucleares do AIDD-Ops é o reconhecimento estrito dos limites da Inteligência Artificial. Modelos de Linguagem (LLMs) são excepcionais para raciocínio heurístico, interpretação semântica e síntese de intenções (processos probabilísticos), mas são inerentemente perigosos se deixados livres para gerar comandos de infraestrutura em tempo real. Em ambientes de produção corporativos, portas, sintaxes de arquivos, certificados de segurança e scripts de banco de dados exigem precisão matemática absoluta.

Regra de Ouro do AIDD-Ops: 'A IA atua na inferência e na escolha da intenção (probabilística); a execução, parametrização e validação são estritamente mecânicas, padronizadas e determinísticas'.

3.1 Decomposição da Squad: Camada Probabilística vs. Motor Determinístico

Agente / Subagente	Bifurcação: Papel Probabilístico (IA) vs. Motor Determinístico (Código)
Agente Coordenador (Lead Solutions Architect)	Probabilística (IA): Interpretação do briefing do cliente

	em linguagem natural, decomposição da dor de negócio em objetivos funcionais. Determinística (Scripts): Orquestração sequencial estrita dos subagentes, coleta estruturada de saídas e montagem do relatório final sem desvios.
Subagente 1 - Discovery & Curadoria de Ferramentas	Probabilística (IA): Compreensão da semântica dos requisitos e busca exploratória no GitHub / registros open source via MCP. Determinística (Regras Rígidas): Filtros booleanos codificados em Python: rejeição automática se <code>dias_sem_commit > 180</code>, rejeição se não houver licença permissiva, validação da existência de Dockerfile oficial.
Subagente 2 - Sizing de Hardware & Camada de Banco Central	Probabilística (IA): Quase nula (apenas identifica quais ferramentas da lista demandam banco relacional). Determinística (Fórmulas & Templates): Cálculo matemático exato: $RAM = (Soma(RAM_mínima) * 1.30)$. Geração do script <code>`init-multiple-databases.sh`</code> a partir de um template Jinja2 padronizado e testado.
Subagente 3 - Borda, DNS & Identidade (Edge & IAM)	Probabilística (IA): Zero. A probabilidade é estritamente proibida nesta camada. Determinística (APIs & MCPs): Payloads tipados enviados à API da Cloudflare via MCP (registros A/CNAME), injeção de labels Traefik pré-fabricadas e scripts Python de criação de clientes OIDC no Authentik.
Subagente 4 - Construtor do Frontend White-Label	Probabilística (IA): Ajuste contextual de textos de interface e harmonização de paleta de cores secundárias a partir do logotipo. Determinística (Scaffold & Compilação): Scaffold base pré-construído em Next.js + TypeScript + Tailwind CSS + shadcn/ui. Injeção estrita do <code>`tenant.config.json`</code> e execução do guardrail de compilação: <code>`tsc --noEmit && next build`</code>.
Subagente 5 - Engenheiro do Gateway Nativo (FastAPI)	Probabilística (IA): Mapeamento semântico de campos de dados entre as ferramentas da stack selecionada. Determinística (Pydantic & Python Async): Estrutura de código baseada em template modular FastAPI assíncrono. Validação estrita de contratos com Pydantic Schemas (rejeição 422 para payloads inválidos).
Guardião Determinístico (Quality & Safety Gates)	Probabilística (IA): Zero. Execução 100% mecânica. Determinística (Scripts & AST): Execução do <code>`docker compose config`</code> (validação sintática), verificação de portas livres, scripts idempotentes de bootstrapping SSH

e testes automatizados de endpoints /healthz com curl.

4. O Pipeline Completo de 10 Fases Ponta a Ponta

Fase 1: Intake & Diagnóstico de Negócio (Probabilística)

Coleta agnóstica em linguagem natural dos objetivos operacionais, dores de processo e volumes de dados esperados.

Fase 2: Curadoria e Seleção da Stack (Híbrida)

Filtro dinâmico de ferramentas open source especializadas para atender à dor informada, com corte determinístico por licença e manutenção ativa.

Fase 3: Sizing e Síntese Dinâmica da Camada de Dados (Determinística)

Dimensionamento matemático das especificações da VPS e mapeamento dos bancos lógicos necessários a partir da seleção de ferramentas da Fase 2.

Fase 4: Bootstrapping e Hardening da VPS via SSH (Determinística)

Conexão SSH automatizada: atualização de pacotes de segurança, instalação do Docker Engine oficial, ativação do UFW firewall e configuração de fail2ban e memória Swap.

Fase 5: Automação de DNS via Cloudflare MCP (Determinística)

Criação programática dos apontamentos A e CNAME para o IP da VPS, ativando proteção contra DDoS e proxy seguro de borda.

Fase 6: Geração de Artefatos, Segredos e Script de Inicialização (Determinística)

Criação determinística de senhas criptográficas fortes (OpenSSL), geração do script ``init-multiple-databases.sh``, montagem do arquivo ``.env`` isolado, parametrização do Traefik e clientes OIDC do Authentik.

Fase 7: Construção do Hub de Integração & Frontend (Híbrida Controlada)

Build do Micro-Serviço Gateway Nativo (FastAPI) e compilação do painel frontend Next.js com validação de tipagem TypeScript.

Fase 8: Deploy Orquestrado em Docker (Determinística)

Validação sintática com ``docker compose config``, montagem dos volumes persistentes, inicialização do banco PostgreSQL centralizado com script ``init-multiple-databases.sh`` e subida subsequente dos microserviços via ``depends_on: condition: service_healthy``.

Fase 9: Bateria de Testes Pré-Produção Pre-Flight (Determinística)

Execução de testes automatizados: checagem de status 200 nos endpoints de saúde (``/healthz``), testes de envio de Webhook simulado e validação de resolução DNS.

Fase 10: Ativação de Backups, Monitoramento & Entrega (Determinística)

Configuração de rotinas automáticas de dump consolidado via `pg\_dumpall` para bucket S3 (Cloudflare R2), subida de monitor de uptime e emissão do relatório final com credenciais master.

5. Análise Comparativa e Decisões de Engenharia

5.1 Monolito vs. Microserviços Modularizados

A tentativa de fundir repositórios heterogêneos (PHP, Ruby, TypeScript, Go) em uma única base de código monolítica gera dívida técnica inviável. A modularização em contêineres Docker com rede interna privada garante isolamento de falhas e preserva atualizações de segurança comunitárias.

5.2 O Padrão Canônico: Micro-Serviço Gateway Nativo (FastAPI) como Escolha Primária

O AIDD-Ops adota como padrão canônico o Micro-Serviço Gateway construído em Python com FastAPI. Consumindo apenas 30 MB a 50 MB de RAM, com tipagem estrita via Pydantic, rotas assíncronas de alta performance e controle de versão limpo no Git, ele se integra organicamente aos Quality Gates em Python do AIDD. O n8n permanece classificado estritamente como um add-on visual opcional para usuários que demandam interface gráfica.

5.3 Imagens Oficiais vs. Dockerfiles Próprios

O agente prioriza imagens oficiais mantidas no Docker Hub ou GHCR. No entanto, para ferramentas modernas em Node.js, Python ou Go que não disponibilizam imagens pré-compiladas, o agente gera automaticamente Dockerfiles multi-stage otimizados (baseados em Alpine ou Distroless). Projetos legados com compilações complexas em C/C++ são descartados por segurança.

5.4 Estratégia de Banco de Dados: Cenário A (Instância Única Centralizada) vs. Anti-Padrão de Tabelas Compartilhadas

Cenário de Banco de Dados	Classificação Arquitetural	Impacto Técnico e Operacional
Cenário A: 1 Instância de SGBD (PostgreSQL Central) com Bancos Lógicos Isolados	CORRETO E ALTAMENTE RECOMENDADO	Sobe-se 1 único contêiner PostgreSQL de alta performance na VPS. Um script de inicialização (`init-multiple-databases.sh`) cria bancos lógicos independentes sob demanda. Economiza de 1 GB a 2 GB de RAM na VPS eliminando múltiplos bancos redundantes e simplifica a rotina de backups.
Cenário B: 1 Único Banco com Tabelas Compartilhadas (Shared Database)	INCORRETO (ANTI-PADRÃO DE ENGENHARIA)	Forçar ferramentas distintas a compartilharem a mesma tabela `users` ou `contacts`. Cada

ferramenta possui seu próprio ORM (Prisma, ActiveRecord, TypeORM) e rotinas automáticas de migração. Modificações manuais quebram o software na primeira atualização (`docker pull`).

5.5 Fluxo Técnico de Provisionamento Dinâmico da Camada de Dados

- **1. Mapeamento Dinâmico de Dependências:** O Subagente 2 analisa a lista de ferramentas selecionadas na Fase 2 e extrai quais necessitam de banco relacional.
- **2. Geração Dinâmica do Script de Inicialização:** Gera o arquivo `init-multiple-databases.sh` que é montado em `/docker-entrypoint-initdb.d/init.sh`. Esse script executa comandos `psql` para criar os bancos lógicos isolados, usuários dedicados e concede privilégios restritos.
- **3. Segregação Rigorosa de Credenciais no .env:** Gera senhas criptográficas individuais para cada serviço e monta as `DATABASE\_URL` segregadas. O isolamento lógico impede que o comprometimento de um serviço afete os dados dos outros.
- **4. Orquestração Determinística com Healthcheck:** No docker-compose.yml, o PostgreSQL executa teste de prontidão (pg_isready -U postgres_admin). Todos os microsservicos dependentes configuram depends_on: postgres: condition: service_healthy.
- **5. Rotina Unificada de Backup (Fase 10):** A rotina de backup executa um único comando consolidado (`pg\_dumpall`), compactando e criptografando todas as bases lógicas da empresa para o bucket S3 (Cloudflare R2).

6. Exemplos Ilustrativos: Matriz de Soluções por Domínio de Negócio

Domínio de Negócio	Stack de Ferramentas Selecionada	Impacto Operacional Direto
Clínicas & Odontologia (Exemplo Saúde)	Typebot (Triagem) + Twenty CRM (Prontuário/Pipeline) + Chatwoot/Evolution API (WhatsApp) + Cal.com (Agendamento)	Redução drástica de no-show (faltas) com lembretes automáticos de 24h e 2h via WhatsApp.
Alimentação & Delivery (Exemplo Varejo)	Typebot (Cardápio Interativo) + Evolution API (WhatsApp) + Odoo POS/Restaurante (KDS/Cozinha) + Listmonk (Retenção)	Eliminação de taxas de marketplaces terceiros e atendimento automático e veloz de pedidos.
Farmácias & Drogarias (Exemplo Recorrência)	Typebot com upload de receitas + Chatwoot com departamentos farmacêuticos + Gateway disparador de reposição de uso contínuo	Fidelização e venda recorrente para pacientes crônicos com alertas preventivos de fim de estoque.

Indústrias & Manufatura (Exemplo B2B)	EspoCRM/SuiteCRM (B2B complexo) + Mautic (Nutrição longa) + Documenso (Assinaturas digitais) + ERPNext (Faturamento)	Rastreabilidade completa de orçamentos técnicos, múltiplos tomadores de decisão e formalização ágil de contratos.
Energia Solar (Exemplo Serviços Técnicos)	Simulador Webstudio/Typebot + Script de cálculo de kWp no Gateway + Twenty CRM + Cal.com (Visitas técnicas) + Documenso	Geração instantânea de pré-proposta e ROI a partir da conta de luz, otimizando o fechamento em campo.

7. Requisitos de Infraestrutura, Segurança e Continuidade

- **Soberania e Privacidade (LGPD/GDPR):** Dados alocados em infraestrutura própria sem compartilhamento com nuvens fechadas de terceiros.
- **Cofre de Segredos (Zero Hardcoding):** Senhas criptográficas geradas em tempo de deploy e injetadas via variáveis de ambiente seguras.
- **Hardening de Rede:** Apenas as portas 80, 443 e 22 (via chave SSH) são abertas; o PostgreSQL central e os microsserviços operam exclusivamente na rede interna Docker.
- **Estratégia de Desastre (Disaster Recovery):** Dumps diários compactados do PostgreSQL central enviados para bucket S3 externo compatível (Cloudflare R2).
- **Observabilidade e Alertas Ativos:** Monitoramento contínuo com Uptime Kuma e Dozzle com notificações automáticas via webhook no WhatsApp/Telegram.

8. Parecer Técnico e Análise de Integração com o Ecossistema AIDD

Esta seção traz o parecer técnico formal sobre a viabilidade de utilizar o repositório heverton-dev/ecossistema-aidd (AI-Driven Development Ecosystem) como motor para operacionalizar a feature agnóstica AIDD-Ops.

8.1 Diagnóstico de Compatibilidade

O ecossistema-aidd fornece a disciplina e o arcabouço de governança agêntica mais rigorosos do mercado: compatibilidade multi-harness (Claude Code, Cursor, Gemini CLI, Grok, Hermes, Antigravity/OpenCode), fonte única em `componentes/` e Quality Gates determinísticos em Python. Ele elimina alucinações de LLMs em tarefas de engenharia.

Por ter sido desenhado inicialmente para código de software local, o AIDD requer a adição de um subprojeto de infraestrutura remota (AIDD-Ops) para orquestrar SSH, Cloudflare MCP, Docker Engine e a camada dinâmica de banco de dados em produção para qualquer stack demandada.

8.2 Mapeamento e Estancamento de Gaps

- **Gap 1: Execução Remota Segura (SSH Runner):** Integração de um executor SSH determinístico baseado em chave pública para bootstrapping da VPS.
- **Gap 2: Conectividade via MCPs (Cloudflare & Docker):** Inclusão no catálogo de componentes dos MCPs oficiais para automação de DNS e controle do daemon Docker.
- **Gap 3: Quality Gate de Docker e Banco (G_INFRA_COMPOSE):** Validação estática de arquivos Compose, portas livres e script de inicialização do PostgreSQL central.
- **Gap 4: Bateria de Testes Pré-Voo (Pre-Flight E2E Tests):** Validação pós-deploy com requisições curl automáticas aos endpoints /healthz e checagem de emissão de TLS.

8.3 Roteiro de Adaptações Arquiteturais no AIDD

1. **Criação de `tools/aidd-ops/`:** Novo subprojeto no diretório `tools/` com CLI `python ecossistema.py ops deploy [ambiente]` e comando canônico `/ops`.
2. **Novos Gates em `gates/`:** `G\_INFRA\_COMPOSE.py` e `G\_PRE\_FLIGHT\_E2E.py` na suíte de auditoria (`ecossistema.py audit`).
3. **Catálogo Canônico de Infraestrutura (`componentes/templates/infra/`):** Templates oficiais agnósticos para Traefik, Authentik e PostgreSQL Central com script dinâmico `init-multiple-databases.sh`.
4. **Extensão do `aidd-generator`:** Suporte à flag `--type=infra-stack` para acionar a squad agêntica de infraestrutura de forma universal para qualquer dor corporativa.

8.4 Conclusão e Veredito

O ecossistema-aidd é o alicerce perfeito para sustentar o AIDD-Ops. A união de sua governança rigorosa com a topologia agnóstica de infraestrutura e orquestração por microserviços viabiliza uma fábrica de software autônoma e universal, capaz de transformar qualquer problema de negócio em um ecossistema operacional de alta performance e soberania absoluta de dados.